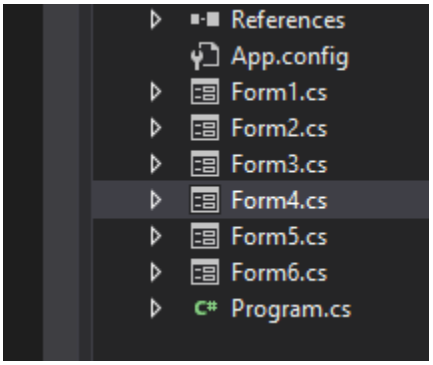
Coding standards used in the code for the application development (Griffindo Toys).

The below demonstrates the different coding Standards used while implementing and coding the griffindo toys payroll system.

Naming Conventions:

- Class Names: Class names follow PascalCase convention, starting with a capital letter (e.g., Form5, Form4, Form3).
- Variable Names: Variable names generally follow camelCase convention, starting with a lowercase letter (e.g., con, empid, name, ms, or, al, mail, add, etc.).
- Control Names: Control names, such as buttons and labels, follow camelCase convention (e.g., button2, button5, button1, dataGridView1).
- Constants: Constants are written in uppercase letters with words separated by underscores (e.g., CONSDATA).




```
if (empid.Text == "" || name.Text == "" || ms.Text == "" || or.Text == "" || al.Text == "" || mail.Text == "" || add.Text == "
{
    MessageBox.Show("Fill all the boxes and try again");

}
else
{

    try
```

Figure 1 - Using Name Conventions

Indentation and Formatting:

The code appears to be properly indented, using white spaces to organize code blocks and nested structures.

```
1 reference
private void button2_Click(object sender, EventArgs e)
{
    if (empid.Text == "" || name.Text == "" || ms.Text == "" || or.Text == "" || a
    {
        MessageBox.Show("Fill all the boxes and try again");

    }
    else
    {

        try
        {


            con.Open();
            string query = "insert into Employee values('" + empid.Text + "','" +
            SqlCommand cmd = new SqlCommand(query, con);
            cmd.ExecuteNonQuery();
            MessageBox.Show("Your Data Has Beeen Successfully Added");
            con.Close();
            filldatagrid();

        }
        catch (Exception Ex)
        {

            MessageBox.Show(Ex.Message);

        }
    }
}

1 reference
private void Form3_Load(object sender, EventArgs e)
{
     filldatagrid();
     LoadData();
}

1 reference
```

Figure 2 - Well Indented Code

Comments and Documentation:

While the code lacks extensive comments, it contains some comments to explain certain sections of code (e.g., try-catch blocks).

```
3 references
private void cdata()
{
    //use to clear the text boxes.
        empid.Clear();
        ms.Clear();
        add.Clear();
        or.Clear();
        al.Clear();
        name.Clear();
        mail.Clear();
```

Figure 3 - Using Documentation and Comments

Code Reusability:

The code does not explicitly demonstrate modularity or code reuse, but it seems to follow a standard Windows Forms application structure.

```
Griffindo_Toys.Form3                                                    cdata()
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using System.Data.SqlClient;

namespace Griffindo_Toys
{
    4 references
    public partial class Form3 : Form
    {
        1 reference
        public Form3()
        {
            InitializeComponent();
        }
        SqlConnection con = new SqlConnection(@"Data Source=DESKTOP-EQH0955\SQLEXPRESS;Initial Catalog=griffindo_toys;Integrated Security=True");
        private DataTable dataTable;
```

Figure 4 - Code Reusability

Error Handling:

The code contains try-catch blocks to catch exceptions and show error messages if any errors occur during database operations.

```
private DataTable dataTable;
1 reference
private void label4_Click(object sender, EventArgs e)
{
    fsfef
}

1 reference
private void button4_Click(object sender, EventArgs e)
{
    Form2 goo = new Form2();
```

Figure 5 - Error Handling

SQL Injection Vulnerability:

The code constructs SQL queries using string concatenation, which can be vulnerable to SQL injection attacks. It's recommended to use parameterized queries or an ORM (Object-Relational Mapping) library to prevent SQL injection.

Figure 6 - SQL Injection Vulnerability